★★ 4. Private Method

Overview

This documentation describes the implementation of private methods in an object-oriented typed language. The implementation extends the language with privacy modifiers that restrict method access while maintaining type safety and allowing for method inheritance.

Feature Requirements

- Private methods can only be called via `this.` or `super.`
- Private methods must be overridden by private methods
- Public methods must be overridden by public methods
- Type safety must be maintained throughout

Implementation Details

1. Method Type Extension

Extended the `MethodT` type to include privacy information:

```
type MethodT
| methodT(arg_type :: Type,
          result_type :: Type,
          body_exp :: ExpI,
          private :: Boolean)
```

2. Syntax Extension

Added support for private method declaration:

```
|'private method $name(arg :: $arg_ty) :: $res_ty: $body':
   values(syntax_to_symbol(name),
          methodT(parse_type(arg_ty),
```

```
                    parse_type(res_ty),
                    parse(body),
                    #true))
```

3. Type Checking Implementation

Method Call Validation

The `typecheck_send` function enforces privacy rules:

```
| methodT(arg_type_m, result_type, body_exp, private):
    if private
    | if is_subtype(arg_type, arg_type_m, t_classes)
      | if is_subclass(caller_class, class_name, t_classes)
        | result_type
        | type_error(caller_type, "private method can only be
called from same class or subclass")
      | type_error(caller_type, to_string(arg_type_m))
    | if is_subtype(arg_type, arg_type_m, t_classes)
      | result_type
      | type_error(caller_type, to_string(arg_type_m))
```

Key enforcement rules:

1. Private method access is restricted to:
   - Calls through `this`
   - Calls through `super`
   - Calls within the same class or subclass
2. Type checking ensures argument types match
3. Return type is preserved for type safety

Override Validation

The `check_override` function ensures privacy consistency:

```
if (methodT.arg_type(method) == methodT.arg_type(super_metho
d)
    && methodT.result_type(method) == methodT.result_type(sup
er_method)
    && methodT.private(method) == methodT.private(super_metho
d))
```

Enforces:

- Private methods must be overridden by private methods
- Public methods must be overridden by public methods
- Type signatures must match exactly

Testing

Comprehensive test cases demonstrate the implementation's correctness:

```
def private_test_classes = [
  values(#'A,
         classT(#'Object,
                [],
                [values(#'secret,
                        methodT(intT(), intT(),
                                plusI(argI(), intI(1)),
                                #true)),  // private
                 values(#'public,
                        methodT(intT(), intT(),
                                sendI(thisI(), #'secret, argI
()),
                                #false))]))  // public
]
```

Test coverage includes:

1. Private method declaration and access

2. Invalid access attempts
3. Override rules
4. Type safety preservation

Design Choices and Rationale

1. Privacy Model

- Chose a simple binary private/public model
- Private access through `this` and `super` only
- Subclass access allowed for better inheritance support

2. Override Rules

- Strict privacy preservation in overrides
- Unlike Java, private methods can be overridden
- Maintains clean separation between public and private interfaces

3. Type System Integration

- Privacy checks integrated into existing type system
- Preserves type soundness
- Clear error messages for privacy violations

Type Safety

The implementation maintains type safety through:

1. **Static Verification**
    - All method calls are checked at compile time
    - Privacy violations are caught before runtime
2. **Override Safety**
    - Privacy modifiers are checked during inheritance

- Type signatures must match exactly

3. **Access Control**
    - Private methods are only accessible in valid contexts
    - Public interface remains consistent

Conclusion

The implementation successfully adds private methods to the language while maintaining type safety and providing clear semantics for method privacy. The design choices create a robust and predictable privacy model that integrates well with the existing type system and inheritance mechanisms.